



**COMSATS University Islamabad,
Sahiwal Campus.**

Software House Management System

A Project Presented To

**COMSATS University Islamabad,
Sahiwal Campus**

In partial fulfillment

of the requirement for the degree of

Bachelor of Science in Software Engineering (2020-2024)

By

Adeel Haider

CIIT/SP20-BSE-003/SWL

Awais Ali

CIIT/SP20-BSE-041/SWL

Danial Rashid

CIIT/SP20-BSE-046/SWL



**COMSATS University Islamabad,
Sahiwal Campus.**

Software House Management System

By

Adeel Haider

CIIT/SP20-BSE-003/SWL

Awais Ali

CIIT/SP20-BSE-041/SWL

Danial Rashid

CIIT/SP20-BSE-046/SWL

Supervisor

Dr Muhammad Farhan

Bachelor of Science in Software Engineering (2020-2024)

The candidate confirms that the work submitted is their own and appropriate credit has been given where reference has been made to the work of others.

DECLARATION

I hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that I have developed this software and accompanied the report entirely based on my personal efforts. If any part of this project is proved to be copied out from any source or found to be a reproduction of some other. I will stand by the consequences. No portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Awais Ali

Danial Rashid

Adeel Haider

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS(SE) “**Software House Management System**” was developed by **Awais Ali (CIIT/SP20-BSE-041/SWL)**, **Danial Rashid (CIIT/SP20-BSE- 046/SWL)** and **Adeel Haider (CIIT/SP20-BSE-003/SWL)** under the supervision of “**Dr. Muhammad Farhan**” and that in his opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Software Engineering.

Supervisor
Dr. Muhammad Farhan
Assistant Professor
Department of computer science
COMSATS University Islamabad, Sahiwal Campus

External Examiner
Dr. Muhammad Munawar Iqbal
Assistant Professor
Department of Computer Science
University of Engineering and Technology, Taxila

Dr. Tariq Ali
Assistant Professor
Head of Department
Department of Computer Science
COMSATS University Islamabad, Sahiwal Campus

Executive Summary

Software House Management System represents a significant step forward in addressing the complex challenges faced by software development companies. This system streamlines various aspects of software house operations, including project management, employee administration, financial tracking, and asset management by providing a comprehensive and integrated solution. The purpose of this project is to manage software house Activities. Software House Management System is a web-based application that is developed using the Django framework, PostgreSQL, and SQLite database. The software will manage all aspects of a software house, including projects, employee roles, attendance records, financial structure, and assets Management. The system will allow for managing multiple projects for different clients and the assignment of roles to employees based on their designation. It will also manage employee attendance records and the financial structure of the software house. Additionally, the system will manage all assets used in the software house. The software can calculate all revenues generated in a month or year and display them in the graphical representation of data. This will make it easier to manage budgets and resources and make decisions.

Acknowledgment

All praise is to Almighty Allah who bestowed upon me a minute portion of His boundless knowledge by which I could accomplish this challenging task.

I am greatly indebted to my project supervisor **Dr. Muhammad Farhan**. Without their supervision, advice, and valuable guidance, the completion of this project would have been doubtful. I am deeply indebted to them for their encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

Awais Ali

Danial Rashid

Adeel Haider

Abbreviations

SRS	Software Require Specification
SDD	Software Design Description
GUI	Graphical User Interface
SDA	Software Design and Architecture
ICT	Introduction to Computer and Technology
SHMS	Software House Management System
SPM	Software Project Management

Table of Contents

1. Introduction.....	1
1.1 Brief.....	1
1.2 Relevance to Course Modules.....	3
1.3 Project Background	4
1.4 Analysis from Literature Review	5
1.5 Methodology and Software Lifecycle for This Project.....	5
1.6 Incremental Model	5
2. Problem Definition.....	7
2. 1 Problem Statement	7
2.2 Deliverables and Development Requirements	7
3. Requirement Analysis.....	10
3.1 Use Case Diagrams	10
3.2 Use Case Description	13
3.3 Functional Requirements.....	16
3.3.1 Module 1: Project Management System	16
3.3.2 Module 2: Role Based System.....	17
3.3.3 Module 3: Attendance Management System	19
3.3.4 Module 4: Financial Management System	20
3.3.5 Module 5: Asset Management System	21
3.4 Non-functional Requirements	23
3.4.1 Performance and Response Time.....	23
3.4.2 Ease of Use	23
3.4.3 Usability.....	24
3.4.4 Security	24
4. Design and Architect.....	25
4.1 System Architecture	25

4.2	Process Flow/Representation	25
4.3	Data Representation	26
4.3.1	Sequence Diagram	26
4.3.2	Class Diagram.....	27
5.	Implementation	29
5.1	Algorithm and Technologies.....	30
5.2	User Interface.....	30
6.	Testing and Evaluation	36
7.	Conclusion and Future Work.....	39
7.1	Conclusion.....	39
7.2	Future Work	39
8.	References.....	41

List of Figures

Figure 1.1:Incremental Model of SHMS	6
Figure 3.1:Use Case Diagram of Customer Console	10
Figure 3.2:Use Case Diagram of Admin Console	11
Figure 3.3:Use Case Diagram of Team Console	12
Figure 4.1: Data Flow Diagram of SHMS	26
Figure 4.2: Sequence Diagram of SHMS	27
Figure 4.3: Class Diagram of SHMS	28
Figure 5.1: Home Screen of SHMS	31
Figure 5.2: Project Dashboard of SHMS	32
Figure 5.3: Login Screen of SHMS	33
Figure 5.4: Services Interface of SHMS	34
Figure 5.5: Team Interface of SHMS	35

List of Tables

Table 1.1:Related System Analysis	5
Table 3.1: Show The Detail Use Case Of View Services.....	13
Table 3.2:Show The Detail Use Case Of Signup.....	13
Table 3.3: Show The Detail Use Case Of Login.	14
Table 3.4: Show The Detail Use Case Of Request Proposal.	14
Table 6.1:Project Management Test Cases	36
Table 6.2: Role Based System Test Cases	36
Table 6.3: Attendance Management Test Cases	37
Table 6.4: Finance Management Test Cases.....	37
Table 6.5: Asset Management Test Cases	37

Chapter 1

Introduction

1. Introduction

Software House Management System is a web base application using Django Framework. In this web application we use PostgreSQL database. This software is useful for managing budget, time, employees, project, teams and their customers. Client can interact to the software developer with the use of this application. Software can manage all the Resources of software house. This Software also calculate all the revenues that was generated in a month or a year and display in graphical representation of data. This software is very helpful for managing software house and resources. In this software we easily manage the attendance of Employees. The following major Functionalities perform in this Software. The system comprises five integral modules, each designed to streamline and enhance various aspects of software house operations. Explore the key functionalities of the Project Management module, the Role-Based System, Finance Management, Assets Management, and Attendance Management. Highlight the significance of this integrated approach in optimizing efficiency, collaboration, and resource utilization within a software house setting. Provide a brief overview of the technology stack employed, emphasizing the system's user-friendly interface and the seamless integration of Django's robust features for a seamless and efficient management experience. [1]

- We Manage all The Projects of different Clients.
- We Assigned roles to different Employees according to their designation.
- The Software Manages all the Employees records and their Attendance.
- It Manages all the financial Structure of the Software House.
- The different Assets used in the Software House Managed by the Software.

1.1 Brief

The main Purpose behind making this software is to provide Online System for COMSATS Software House. The Record of COMSATS Software House Will be maintained by using this Software. we have getting hands on Experience of Full Stack Web Developer with Django as a Backend and Bootstrap for front End with PostgreSQL Database. The core benefit of software House management system is automating project planning and scheduling and manage all Software House. The Software Manages all the Employees records and their Attendance. We Manage all The Projects of different Clients. It Manages all the financial Structure of the Software House. The different Assets used in the Software House Managed by the Software.

- In the fast-evolving landscape of software development, the Software House Management System stands as a beacon of operational efficiency and strategic prowess. Crafted on the robust Django framework, this web application addresses the quintessential needs of a software house through five pivotal modules: Project Management, Role-based System, Finance Management, Assets Management, and Attendance Management.
- Efficient project management is the linchpin of success in software development, dictating the timely delivery and quality of products. The Software House Management System's Project Management module ensures a streamlined approach, guiding teams through intricate project lifecycles with precision.
- The Role-based System establishes a collaborative environment by facilitating the seamless assignment of roles, aligning team members with tasks that complement their skills and expertise. This dynamic allocation enhances teamwork and maximizes individual contributions.
- Financial management is paramount in the business of software development. The Finance Management module provides a comprehensive toolkit for budgeting, tracking expenses, and managing revenue streams, empowering software houses to make informed financial decisions.
- Assets Management is the guardian of technological resources, optimizing infrastructure to ensure peak performance. By meticulously overseeing hardware and software assets, this module enhances the overall efficiency and longevity of the technology stack.
- In the realm of human resources, the Attendance Management module emerges as the heartbeat of workforce governance. By introducing transparency and discipline, it not only streamlines administrative processes but also contributes to a culture of accountability.

In essence, the Software House Management System is a strategic amalgamation of these modules, each playing a crucial role in fortifying the foundations of a software development organization. Beyond mere functionality, this web application embodies a commitment to operational excellence, aiming to streamline processes, enhance productivity, and propel software houses toward unparalleled success in an ever-evolving industry. Welcome to a future where the synergy of technology and management converges to redefine the dynamics of software house operations. [1]

1.2 Relevance to Course Modules

The academic foundation of the Software House Management System project is rooted in a diverse range of subjects, each contributing significantly to its conceptualization, design, and implementation. The following key subjects have played a pivotal role in shaping the project's scope, functionality, and overall success.

Website Development: This subject leads the groundwork for the practical aspects of building a web application. Concepts of front-end and back-end development, user interface design, and web technologies were instrumental in crafting the user-centric interfaces and responsive design of the Software House Management System.

Database: A solid understanding of database management is critical for any management system. Knowledge in this area influenced the design and implementation of the database schema for storing and retrieving data efficiently across various modules of the system, ensuring data integrity and security.

Software Project Management: This subject provided insights into project planning, execution, and control. Principles of project management guided the systematic development and timely delivery of the Software House Management System. Concepts such as task scheduling, resource allocation, and risk management were applied throughout the project lifecycle.

Software Design and Architecture: Understanding software design principles and architectural patterns was crucial in crafting a scalable and maintainable system. These subjects influenced decisions related to the system's overall structure, module interdependencies, and adherence to best practices for robust software architecture.

Object-Oriented Programming: The project leveraged object-oriented programming principles to enhance code modularity, encapsulation, and reusability. Concepts such as classes, objects, and inheritance were instrumental in designing the modular architecture of the Software House Management System.

Introduction to Software Engineering: Theoretical foundations of software engineering, including software development methodologies and lifecycle models, guided the systematic approach to the project. This subject influenced requirement analysis, system design, coding, testing, and maintenance phases, ensuring a comprehensive software engineering perspective.

Human-Computer Interaction: A user-friendly interface is paramount for the success of any management system. Insights from Human-Computer Interaction facilitated the design of an intuitive and ergonomic user interface, enhancing user experience and fostering user

acceptance across the diverse functionalities of the Software House Management System.

Programming Fundamentals: A strong grasp of programming fundamentals, including algorithmic thinking and problem-solving skills, was essential for coding the various functionalities of the system. This subject influenced the efficiency and effectiveness of the codebase, ensuring optimal performance and reliability.

Introduction to Computer and Technology: Fundamental concepts in computer science and technology provided the necessary background for understanding the underlying principles of the Software House Management System. This subject laid the foundation for students to comprehend the technical aspects of the project, fostering a holistic approach to its development.[2]

1.3 Project Background

The impetus behind the development of the Software House Management System (SHMS) stems from the evident need within our Software House for a dedicated software solution to manage the myriad projects undertaken. Currently, the absence of a specialized software product has compelled the manual recording of software house activities, a process that proves laborious and prone to inefficiencies.

Recognizing this gap, our team has chosen the Software House Management System as the focus of our Final Year Project (FYP). The overarching goal is to introduce an innovative and tailored solution that addresses the challenges posed by the manual management of diverse projects within the software house. SHMS is envisioned as the cornerstone for transforming and modernizing our software house's operational landscape.

By opting for SHMS as our FYP, we aim to revolutionize the way our software house handles project management, resource allocation, financial tracking, and overall administration. This project background underscores the pressing need for an automated system that not only centralizes project records but also introduces efficiency and accuracy to the software house's day-to-day operations. SHMS, as the chosen FYP, embodies our commitment to bridging this technological gap and ushering in a new era of streamlined and optimized software house management.

1.4 Analysis from Literature Review

Table 1.1 Related System Analysis

Application Name	Weakness	Proposed Project Solution
Jira Software	Jira is used for Software Project Management. It does not have records about Finance, Asset and Attendance Management record.	Software House Management System will manage all the record of Finance, Asset and Attendance.

1.5 Methodology and Software Lifecycle for This Project

This system will be developed into many development ways, but most authentic and suitable life cycle is Incremental model. The incremental build model is a method of software development where the product is designed, implemented, and tested incrementally until the product is finished. It involves both development and maintenance. The product is defined as finished when it satisfies all its requirements. Design methodology is the broader study of method in design: the study of the principles, practices, and procedures of designing. A software process model is the mechanism of dividing software development work into distinct phases to improve design, product management, and project management. It is also known as a software development life cycle.

1.6 Incremental Model

In the context of the Software House Management System (SHMS), the Incremental Model represents a strategic approach to software development that aligns seamlessly with the project's dynamic requirements. In adopting this model, the expansive set of requirements for SHMS is systematically partitioned into standalone modules, each constituting a distinct component of the software development cycle.

In the initial phases, each module undergoes the sequential stages of requirements analysis, design, implementation, and testing. This incremental development allows for a focused and iterative approach, where the emphasis is placed on enhancing specific functionalities with each module release. Consequently, every subsequent release augments the existing features,

gradually evolving the software to meet the comprehensive needs of the Software House Management System [3].

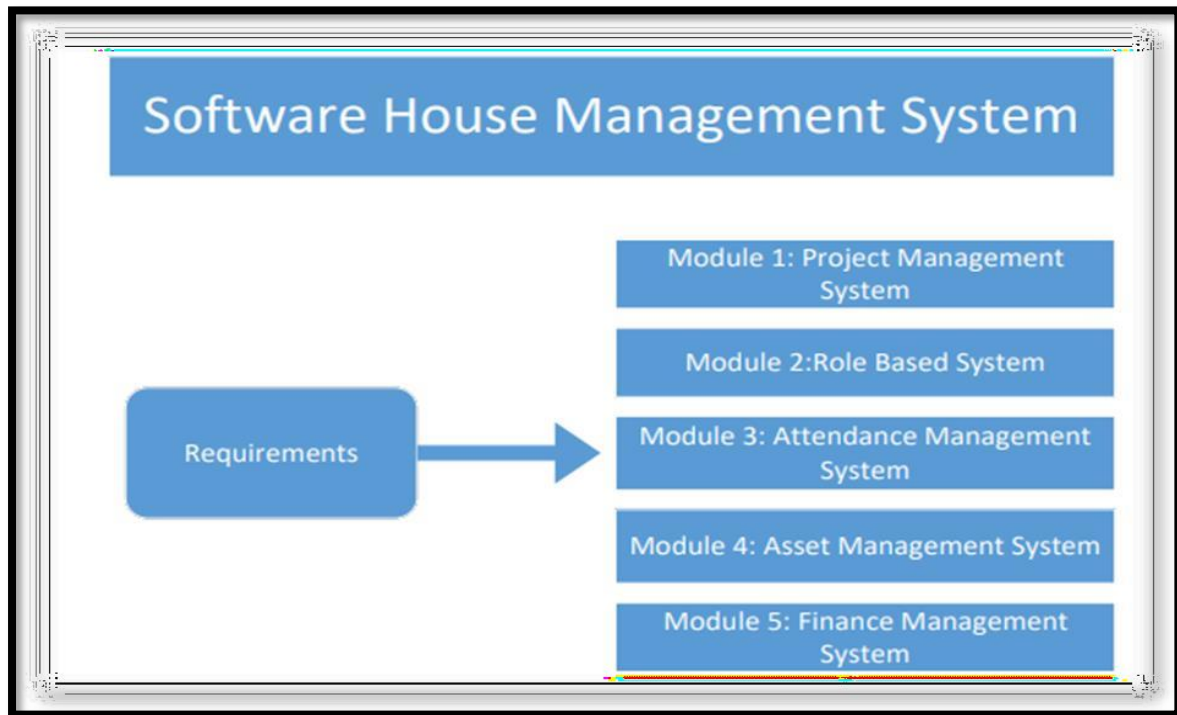


Figure 1.1 Incremental model of SHMS

Chapter 2

Problem Definition

2. Problem Definition

2.1 Problem Statement

The problem statement for the Software House Management System (SHMS) revolves around the complexities faced by a software development company in managing its operations. The current challenges include inefficient project management, fragmented employee administration, imprecise financial tracking, and a lack of streamlined asset management. In response to these issues, the SHMS is conceived as a comprehensive and integrated solution. The SHMS aims to tackle inefficiencies by providing a suite of tools and features that optimize project workflows. It seeks to simplify employee administration by centralizing records and attendance tracking. Moreover, the system addresses the need for accurate financial tracking, offering insights into the financial health of the software house. This includes managing budgets, calculating revenues, and presenting financial data in a graphical format for informed decision-making.

Asset management is another critical aspect that SHMS focuses on, ensuring the efficient utilization of resources within the software house. By offering a centralized platform for project, employee, financial, and asset management, the system aims to enhance overall operational efficiency, foster transparency, and provide a solution tailored to the unique challenges faced by the software development company. The SHMS project aims to transform the way the software house manages its activities, mitigating challenges and promoting a more streamlined and effective management approach. [4]

2.2 Deliverables and Development Requirements

Web Application: The core deliverable is a web-based application accessible through standard web browsers. This ensures platform independence, enabling users to interact with the system seamlessly from various devices. Provides a centralized and accessible platform for users to manage projects, employees, finances, assets, and other critical aspects of software house operations. Responsive design, cross-browser compatibility, and a user-friendly interface for a positive user experience.

User Roles: Different user roles (e.g., admin, project manager, employee) are defined, each with specific access rights and permissions tailored to their responsibilities. Ensures data security and confidentiality by limiting access to sensitive information only to authorized users. Role-based access control, permission management, and a clear hierarchy of user roles.

Database: PostgreSQL database is used to store and manage project data, employee records, financial information, and asset details. Serves as the central repository of critical information, facilitating efficient data retrieval, storage, and management. Database schema design, data normalization, and indexing for optimal performance.

Project Management: The system includes features for project creation, deadline setting, task assignment, and progress tracking. Enhances project efficiency by providing tools for planning, monitoring, and managing the entire project lifecycle. Project creation wizard, Gantt charts for timeline visualization, task assignment, and progress tracking.

Employee Management: Tools are provided for adding and managing employee records, tracking attendance, defining roles and responsibilities, and assessing employee performance. Streamlines HR-related tasks, facilitates workforce management, and supports performance evaluation. Employee record management, attendance tracking, role assignment, and performance assessment tools.

Financial Module: The system includes a dedicated module for recording financial data, including income, expenses, and generating financial reports. Supports informed financial decision-making by providing insights into the financial health of the software house. Income and expense recording, financial reporting tools, budgeting features.

Asset Tracking: The system should have functionality for maintaining an inventory of software house assets, including hardware, software licenses, and their status. Ensures optimal resource utilization, tracks asset status, and facilitates informed decision-making regarding resource allocation. Asset inventory management, status tracking, and software license tracking.

Security: Robust security measures are implemented to protect sensitive data and user access. Ensures the confidentiality and integrity of information, safeguards against unauthorized access, and prevents data breaches. Secure user authentication, encryption of sensitive data, role-based access controls.

Reporting: The system should allow users to generate various reports related to projects, finances, and employee performance. Provides valuable insights for decision-making, performance evaluation, and overall software house management. Customizable reporting tools, predefined report templates, data visualization.

User Interface: An intuitive and user-friendly interface is essential for easy navigation and interaction. Improves user adoption and satisfaction by providing a positive and efficient user experience. Responsive design, clear navigation, visually appealing layouts, and an intuitive

dashboard.

Documentation: Comprehensive documentation is provided for users and developers to guide them in system usage, installation, and administration. Facilitates a smooth onboarding process, assists users in utilizing system features, and guides developers in system maintenance. User manuals, installation guides, API documentation, and administrative guides. [5]

Chapter 3

Requirement Analysis

3. Requirement Analysis

3.1 Use Case Diagrams

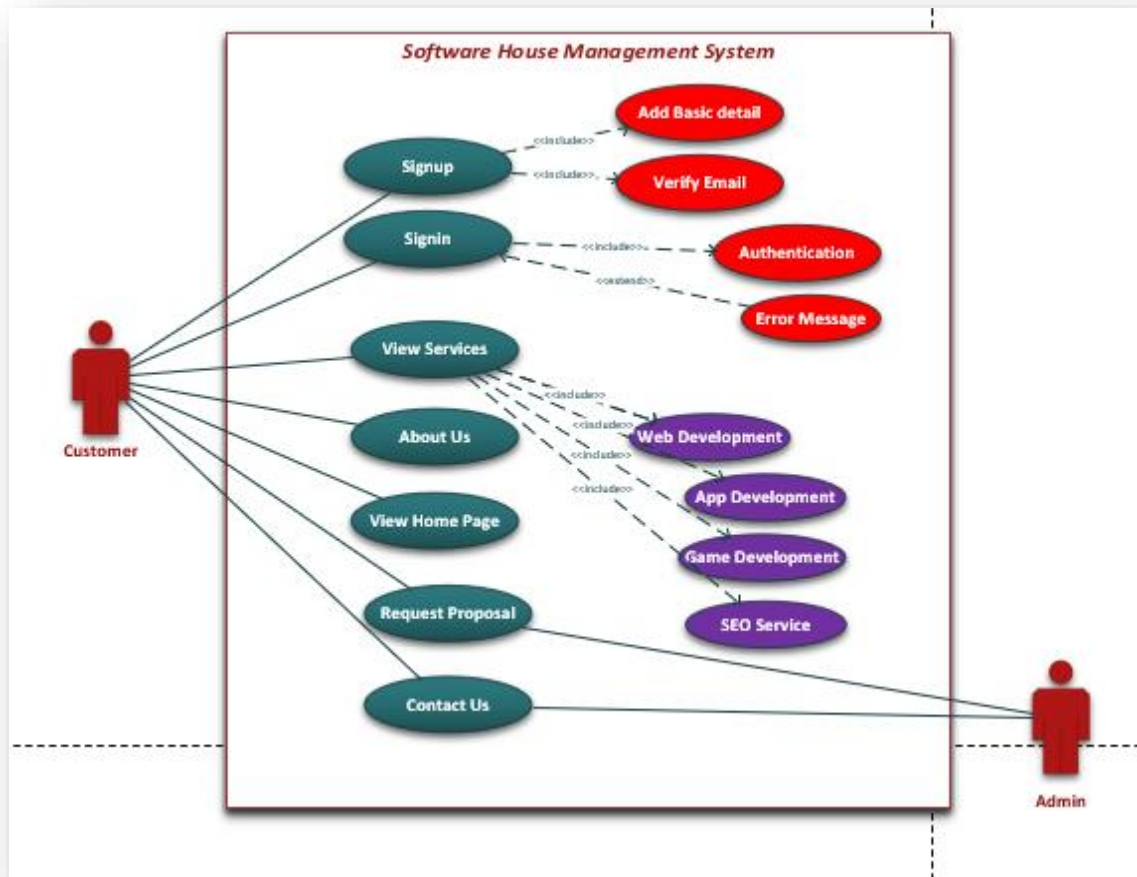


Figure 3.1: Use case Diagram of Customer Console

The Software House Management System website offers a streamlined customer journey starting with an engaging homepage, leading users through an intuitive signup process to submit service requests. Upon login, customers access a personalized dashboard displaying current service status and project milestones. The integrated communication system provides real-time updates, fostering direct interaction with the software house. The platform facilitates detailed service monitoring, offering access to project documentation and feedback options. A resource center and account management tools enhance user experience, while a secure logout option and optional feedback survey complete the seamless and user-friendly process, encouraging customers to return for future services.

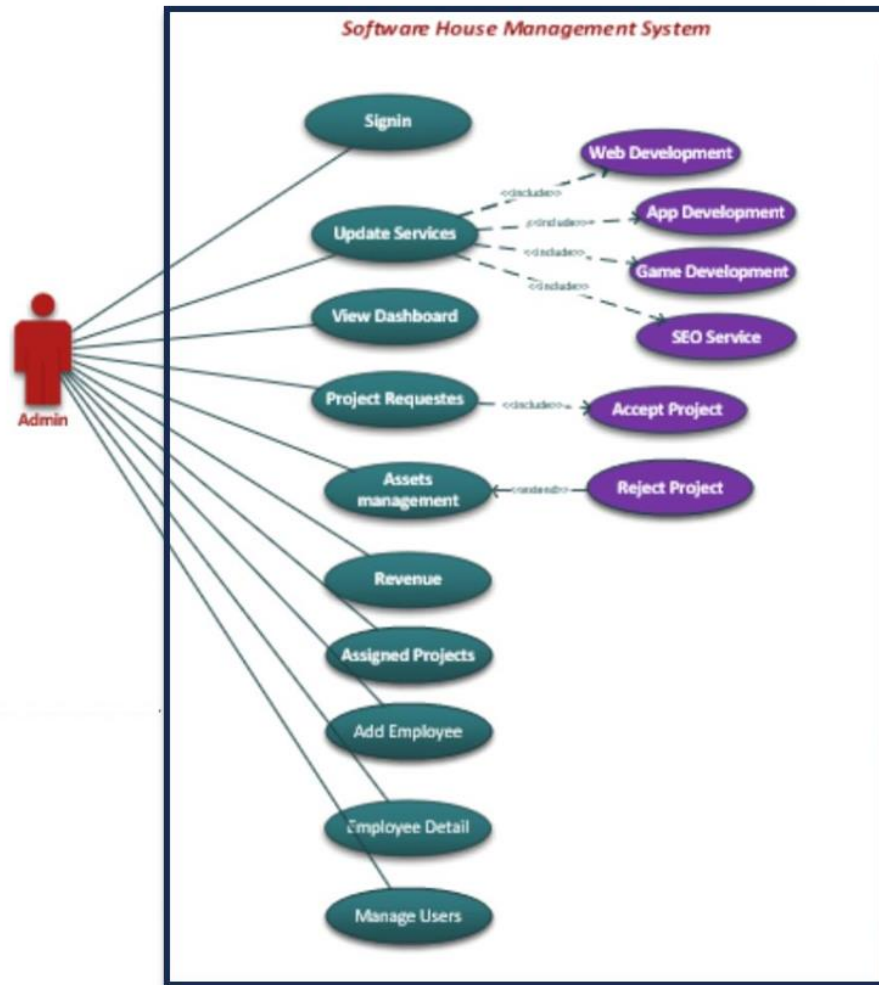


Figure 3.2: Use case Diagram of admin Console

After a secure login of the Admin, the dynamic dashboard presents metrics like pending service requests, asset status, ongoing projects, user activity, and revenue details. Service Management allows the admin to review, approve/reject service requests, and assign tasks. Asset Management provides a streamlined module to view, track, and update software/hardware assets, assign them to projects/users. Project Management monitors ongoing projects, including timelines, milestones, task assignments, and team management. User Management enables account creation, role updates, and activity tracking. Revenue Details offer insights into generated revenue, detailed reports, payment tracking, and invoice management. The Settings panel empowers the admin to customize configurations, and Logs/Reports provide a comprehensive system overview for troubleshooting and auditing. Secure logout ensures data integrity and user authentication, ensuring efficient control over the software house's operation.

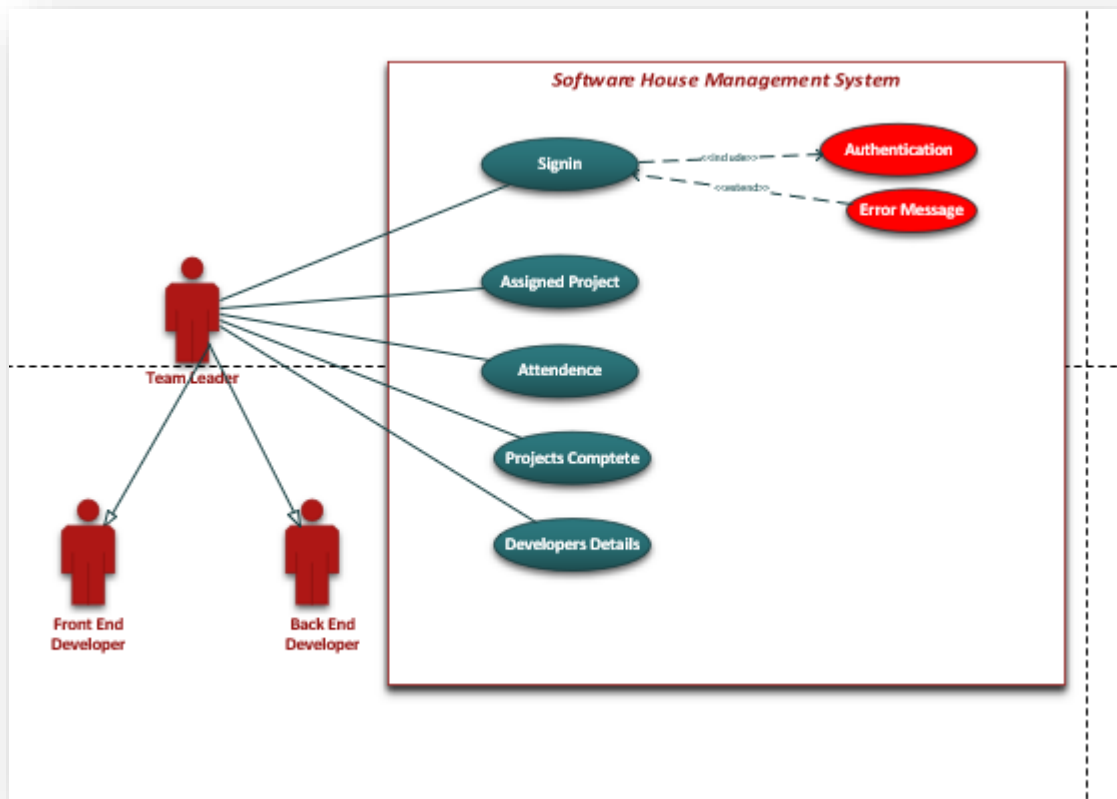


Figure 3.3: Use case Diagram of Team Console

In the Software House Management System, the Team Leader initiates the workflow by logging in and gaining access to a personalized dashboard, offering a comprehensive overview of key metrics, project statuses, and team performance indicators. Navigating to the "Project Management" section, the Team Leader can both view existing projects and create new ones, specifying details such as project name, description, deadlines, and priority. Team members are assigned based on skills and availability, facilitated by a user-friendly interface. Detailed project information, including timelines and milestones, is accessible, aided by Gantt charts. Task allocation and real-time collaboration are integral, with the Team Leader monitoring progress through visual indicators and reports. Resource management tools allow adjustments to team member allocations, and regular reviews with feedback mechanisms ensure project and team efficiency. Upon project completion, the Team Leader can mark it as finished, and the system archives relevant data, offering an option to generate comprehensive project lifecycle reports. This integrated flow optimizes project management and team coordination within the system.

3.2 Use Case Description

Table 3.1: Show the detail use case of View Services

Use Case ID:	01
Use Case Name:	View Services
Actors:	Primary Actor: Customer Secondary Actors: Admin
Description:	The Customer Open website of Software House Management System by Entering URL of Website. The customer view Home page and see all the Services provided by admin. After selecting the required service, the Customer fills the request form. The admin may accept or reject the request form.
Trigger:	By Using Sign up page user can register themselves. After registration They can request for services.
Preconditions:	User must sign up by providing personal information. User must be login by providing proper email and password.
Postconditions:	User may visit Home page of the Website. User may fill request proposal for service. Admin may accept or reject request proposal.
Normal Flow:	User can login into website. User can see different development Services. User can submit request form.
Alternative Flows: [Alternative Flow 1 – Not in Network]	User request for Service. Admin may Accept request. Admin may reject request.

Table 3.2: Show the detail use case of Signup

Use Case ID:	02
---------------------	----

Use Case Name:	Signup
Actors:	Primary Actor: User Secondary Actors: Admin
Description:	Allows a user to create a new account.
Trigger:	User wants to register for a new account.
Preconditions:	The user does not have an existing account with the system.
Postconditions:	User successfully registers an account.
Normal Flow:	User accesses the signup page. User enters their registration details (e.g., username, email, password). User clicks the "Signup" button. System validates the input data. If the data is valid, the user account is created, and the user is redirected to the login page. If there are errors in the input data, appropriate error messages are displayed.
Alternative Flows: [Alternative Flow 1 – Not in Network]	If the chosen username or email is already in use, the system will prompt the user to choose a different one.

Table 3.3: Show the detail use case of login.

Use Case ID:	03
Use Case Name:	Login
Actors:	Primary Actor: User Secondary Actors: Admin
Description:	Allows a user to log into the system.

	items, and places an order for a meal to be picked up in the cafeteria or delivered to a specified location within a specified 15-minute time window.
Trigger:	User wants to access the system and must log in.
Precondition:	The user must have a registered account.
Postcondition:	User gains access to the system.
Normal Flow:	User accesses the login page. User enters their username and password. User clicks the "Login" button. System validates the credentials. If the credentials are valid, the user is logged in and granted access. If the credentials are invalid, an error message is displayed.
Alternative Flows: [Alternative Flow 1 – Not in Network]	If the user forgets their password, they can request a password reset.

Table 3.4: Show the detail use case of Request Proposal.

Use Case ID:	04
Use Case Name:	Request Proposal
Actors:	Primary Actor: User Secondary Actors: Admin
Description:	Allows a user to request a project proposal.
Trigger:	User wants to initiate a project proposal request.
Precondition:	User is logged in.

Postcondition:	A project proposal request is created in the system.
Normal Flow:	User logs in to the system. User navigates to the "Request Proposal" section. User fills in the necessary project details (e.g., project name, description, requirements). User submits the proposal request. System validates the request data. If the request is valid, it is saved in the system, and the user receives a confirmation. If there are errors in the request data, appropriate error messages are displayed.
Alternative Flows: [Alternative Flow 1 – Not in Network]	If the user decides to cancel the request, they can abort the process at any time. If the user has previously submitted a similar request, the system may suggest alternatives or allow for editing the existing request.

3.3 Functional Requirements

3.3.1 Module 1: Project Management System

The Project Management System provides a comprehensive platform for administrators to oversee, manage, and optimize the entire project lifecycle, from assignment to completion, fostering collaboration and ensuring that projects are delivered successfully and on time.

- In the Project Management System, the administrator plays a pivotal role in overseeing and managing all projects within the organization. The administrator has the authority to create, modify, and control access to projects, ensuring a centralized point of control.
- The system facilitates the allocation of projects among various Software Development teams. This allocation is based on factors such as team expertise, workload, and project requirements. Efficient assignment ensures that projects are distributed effectively and align with the capabilities of each development team.

- The administrator maintains comprehensive records of both Completed and In-Progress Projects. This involves updating project status, tracking milestones, and managing project timelines. The system provides a real-time view of the project landscape, aiding in decision-making and resource allocation.
- One of the key responsibilities of the administrator is to monitor the working status of different projects, keeping a close eye on their progress relative to deadlines. This involves regular checks on project development, identifying potential bottlenecks, and ensuring that the teams are on track to meet project goals.
- The system incorporates features for setting and managing project deadlines. Administrators can define project timelines, milestones, and delivery dates. Monitoring tools provide visibility into the progress of tasks, allowing timely interventions to ensure that projects are completed within specified timeframes.
- The Project Management System fosters effective communication and collaboration among project teams and the administrator. It may include features such as messaging, notifications, and document sharing to facilitate seamless information flow, ensuring that all stakeholders are informed and aligned.
- The system supports performance analysis by providing data on project completion rates, adherence to deadlines, and team productivity. This information is valuable for evaluating the efficiency of processes, identifying areas for improvement, and making informed decisions to enhance overall project management.
- Administrators can efficiently allocate resources based on project requirements and team capabilities. This involves assigning personnel, tools, and other necessary resources to optimize project development and ensure that teams have what they need to succeed.
- The Project Management System incorporates security features to safeguard sensitive project information. Access control mechanisms ensure that only authorized personnel can view or modify specific project details, maintaining confidentiality and integrity.

3.3.2 Module 2: Role Based System

A role-based system in Software House Management ensures a well-organized and specialized

workforce, fostering collaboration and efficiency in delivering high-quality web and app development projects. Each team and role contribute to the overall success of the organization by leveraging their expertise in a coordinated manner.

- The Software House adopts a role-based system to organize its workforce efficiently. Different teams are established based on specific roles and responsibilities within the software development process. This hierarchical structure ensures clarity in task assignments and promotes specialization.
- One of the specialized teams within the Software House is the Web Development Team. This team is tasked with handling projects related to web development. Its members possess skills and expertise specifically tailored for designing, developing, and maintaining web applications.
- The Software House also includes an App Development Team, which is dedicated to the creation of mobile applications. This team is equipped with the necessary skills to tackle the unique challenges associated with app development, ensuring that projects meet the standards and requirements of the mobile platform.
- Each development team may be further subdivided based on specific roles, such as Front-End Developers, Back-End Developers, and Database Engineers. This division ensures that individuals with distinct skill sets collaborate harmoniously to achieve the overall objectives of a project.
- Front-End Developers focus on creating the user interface and experience of applications. They are responsible for designing and implementing features that users interact with directly, ensuring a seamless and visually appealing user experience.
- Back-End Developers specialize in server-side development, managing databases, and handling the logic and functionality that occur behind the scenes. They work to ensure the proper functioning of the application and manage data storage and retrieval.
- Database Engineers are dedicated to the design, implementation, and maintenance of databases. They focus on data architecture, optimization, and security to ensure that the application's data is efficiently managed and remains secure.
- While teams are specialized in their respective domains, collaboration across teams is essential. Cross-functional collaboration ensures that the front-end, back-end, and database components seamlessly integrate to deliver a fully functional and cohesive software solution.

- The role-based system enables efficient project assignment based on the required skill sets. The Software House management assigns projects to teams and individuals based on their roles, ensuring that the right combination of skills is applied to each project.
- The role-based system encourages ongoing skill development and training within each specialized team. Team members have the opportunity to enhance their expertise in their specific roles, keeping up with industry trends and best practices.
- The organizational structure allows for flexibility and adaptability, enabling the Software House to easily scale or reorganize teams based on project requirements, technological advancements, or changes in market demands.

3.3.3 Module 3: Attendance Management System

The Attendance Management System in the Software House Management framework provides a systematic and efficient way to monitor and record employee attendance. By leveraging technology and automation, it contributes to workforce management, payroll accuracy, and adherence to organizational policies.

- The Attendance Management System within the Software House operates on a two-times-a-day attendance marking system. Employees are required to register their attendance twice during the working day, typically at the beginning (morning) and at the end (evening). This helps in accurately tracking the presence and working hours of each employee.
- Employees mark their attendance by logging into the centralized Attendance Management System. This can be done through a dedicated software application, a web-based platform, or any other system specified by the Software House. The login process serves as a timestamp for attendance tracking.
- To enhance security and prevent attendance fraud, the system may incorporate biometric (such as fingerprint or facial recognition) or card-based authentication. This ensures that attendance records are accurate and reliable, minimizing the risk of proxy attendance.
- The system provides real-time attendance tracking, allowing administrators and managers to monitor employee attendance as it happens. This feature provides immediate visibility into who is present in the office and who is not, facilitating better workforce management.
- The Attendance Management System generates automated attendance reports based on the data collected throughout the day. These reports may include information such as daily attendance

summaries, late arrivals, early departures, and total working hours.

- For employees working beyond regular hours, the system may include overtime tracking functionality. This ensures accurate recording of extra hours worked, enabling proper compensation and adherence to labor regulations.
- The Attendance Management System may integrate with the leave management module of the broader Software House Management System. This integration helps in reconciling attendance data with approved leaves, ensuring accurate and comprehensive workforce records.
- The system can be configured to send notifications and alerts to employees for attendance marking reminders. Additionally, managers and administrators may receive alerts for any irregularities or exceptions in attendance, allowing for prompt intervention if needed.
- The Attendance Management System is designed with an accessible and user-friendly interface, making it easy for employees to log in and mark their attendance. This simplicity encourages consistent and accurate attendance recording.

3.3.4 Module 4: Financial Management System

The Financial Management System in the Software House Management framework serves as a vital tool for administrators to monitor, analyze, and manage the financial aspects of the organization. By providing real-time financial insights and supporting strategic decision-making, the system contributes to the overall efficiency and sustainability of the Software House.

- The Financial Management System within the Software House integrates a feature where comprehensive financial statements are readily available on the admin dashboard. This strategic placement ensures that administrators have immediate access to crucial financial information without the need for extensive navigation.
- The core calculation within the Financial Management System revolves around the total revenue generated by all the projects undertaken by the Software House. This involves aggregating income streams from various projects to provide an overarching view of the financial health of the organization.
- The system allows for detailed tracking of revenue generated by individual projects. This level of granularity is essential for understanding the financial performance of each project, identifying successful endeavors, and pinpointing areas that may require attention.

- In addition to revenue, the Financial Management System incorporates features for tracking and managing expenses. This includes project-related costs, operational expenditures, and any other financial outflows. Accurate expense tracking contributes to precise financial reporting and budget management.
- The system generates Profit and Loss statements, offering insights into the financial gains and losses incurred by the Software House. This analysis aids in strategic decision-making, allowing administrators to identify profitable ventures and areas where cost optimization may be necessary.
- Financial planning is supported through budgeting and forecasting tools. The system assists in creating budgets for projects, departments, or the organization as a whole. It also provides forecasting capabilities, helping stakeholders anticipate future financial scenarios based on historical data and current trends.
- The Financial Management System manages the invoicing process for projects, ensuring timely and accurate billing. Revenue recognition protocols are applied, aligning with accounting standards to appropriately record revenue based on the completion of project milestones or deliverables.
- Administrators have the flexibility to customize financial reports based on specific parameters, timeframes, or organizational requirements. Customization options enhance the relevance and usability of financial data for decision-makers.
- The Financial Management System is seamlessly integrated with the Project Management module. This integration allows for a holistic view of the financial impact of each project, aligning financial data with project timelines, milestones, and completion statuses.
- To safeguard sensitive financial information, the system employs robust security measures and access controls. This ensures that only authorized personnel have access to financial data, protecting the integrity and confidentiality of financial records.

3.3.5 Module 5: Asset Management System

The Asset Management System in the Software House Management framework is a comprehensive tool that enables administrators to effectively manage, monitor, and optimize the entire lifecycle of assets. By providing real-time visibility and control, the system contributes to improved asset utilization, cost efficiency, and overall organizational productivity.

- The Asset Management System serves as a centralized repository where all assets owned by the Software House are recorded. This includes physical assets such as computers, servers, and office equipment, as well as intangible assets like software licenses.
- The system provides administrative control to the administrator, allowing them to perform key functions such as adding new assets, deleting obsolete ones, updating information, and viewing a comprehensive list of all assets. This level of control ensures that the asset database remains accurate and up-to-date.
- Assets are categorized within the system for efficient organization and retrieval. Categories may include hardware, software, furniture, and other relevant classifications. This categorization aids in quick identification and management of different types of assets.
- Each asset is assigned a unique identifier within the system. This identification method facilitates precise tracking, eliminating confusion and ensuring that the system accurately reflects the current status and location of each asset.
- The system supports asset lifecycle management, covering the entire lifespan of each asset from acquisition to disposal. This involves tracking asset depreciation, maintenance schedules, and other relevant events throughout the asset's existence.
- Maintenance schedules for physical assets are integrated into the Asset Management System. This feature allows administrators to schedule and track routine maintenance activities, ensuring that assets remain in optimal condition and minimizing unexpected downtime.
- For assets that may be moved within the organization, the system tracks their movement history. This includes information on when an asset was relocated, who initiated the move, and the new location. This tracking capability aids in inventory management and prevents loss or misplacement of assets.
- The Asset Management System is seamlessly integrated with the Financial Management module. This integration ensures that the value of assets is accurately reflected in financial statements, and any financial transactions related to assets, such as acquisitions or disposals, are recorded appropriately.
- For tangible assets subject to depreciation, the system calculates depreciation over time. This calculation is essential for financial reporting, allowing the organization to account for the reduction in the value of assets over their useful life.

- The system generates reports and analytics related to assets. These reports may include asset utilization, maintenance history, depreciation summaries, and other relevant metrics. Analytical insights assist in making informed decisions about asset management and investment.
- Security measures and access controls are implemented to protect sensitive asset information. Only authorized personnel have access to certain functionalities, ensuring that asset data is secure and confidential.

3.4 Non-functional Requirements

The non-functional requirements for the software house management system emphasize the importance of performance, user-friendliness, usability, and security. These aspects collectively contribute to the overall effectiveness, reliability, and security of the system, ensuring a positive experience for users and safeguarding sensitive information.

3.4.1 Performance and Response Time

The software house management system should demonstrate a high-performance rate when executing user inputs. This involves efficient processing of commands and tasks, minimizing delays, and ensuring a responsive experience for users. The system must offer swift feedback or responses to user actions. This includes prompt loading of pages, quick execution of commands, and minimal latency in interactions. The objective is to enhance user satisfaction by providing a seamless and responsive user experience.

3.4.2 Ease of Use

The software house management system should feature a user-friendly interface that is intuitive and easy to navigate. This design should cater to users with varying levels of understanding, ensuring that the system is accessible to both novice and experienced users. The user interface should be designed in a way that minimizes the need for extensive training. Users should be able to comprehend the system's functionality with ease, reducing the learning curve and enabling them to efficiently use the software without specialized training.

3.4.3 Usability

Users should feel comfortable interacting with the software without requiring specialized training. The system should provide clear cues and guidance on further actions, ensuring that users can navigate through different modules and functionalities without ambiguity. Web applications within the software house management system should have a straightforward and unambiguous presentation of functionality. Users should be able to understand the purpose and operation of each feature, fostering a sense of confidence and control in their interactions.

3.4.4 Security

The system should implement robust authorization mechanisms to control access to different modules and functionalities. Only authorized personnel should have access to sensitive information and system features. Secure authentication protocols should be in place to verify the identity of users accessing the system. This ensures that only legitimate users with proper credentials can log in and use the software. Information transmission and storage should be secured through encryption protocols. This includes encrypting data during communication over networks and ensuring that sensitive data at rest is protected from unauthorized access. Effective session management should be implemented to monitor and control user sessions. This involves mechanisms to handle user logins, logouts, and session timeouts to prevent unauthorized access to the system. The system should be equipped with robust exception management capabilities. This involves handling errors and exceptions gracefully, providing informative error messages, and preventing the exposure of sensitive information in error scenarios.

Chapter 4

Design and Architecture

4. Design and Architect

4.1 System Architecture

The Software House Management System's architecture is a well-structured, modular framework that prioritizes scalability, flexibility, and performance. It seamlessly integrates various components to provide a comprehensive solution for managing service requests, projects, and client communication within the software house. The System Architecture of the Software House Management System is meticulously crafted for optimal functionality and user experience. Comprising a presentation layer ensuring intuitive client interactions, an application layer managing business logic and service processing, and specialized modules for service requests, project management, and communication, the system operates on a modular and scalable framework. Robust user authentication, data encryption, and a relational database system secure sensitive information. Seamless integration capabilities with external systems, coupled with a responsive design for different devices, enhance the system's versatility. The architecture's scalability accommodates increased demand and evolving functionalities while its modular structure facilitates easy maintenance and upgrades. Performance optimization measures, such as efficient database queries and monitoring tools, ensure a reliable and responsive Software House Management System that adeptly manages service requests, projects, and client communications.

4.2 Process Flow/Representation

The Software House Management System employs a well-defined data flow to enhance the customer journey. The process initiates with users interacting with an engaging homepage, leading to a signup module. This triggers a data flow for user registration and service request submission. Once logged in, customers access a personalized dashboard, involving data flows for retrieving current service status and project milestones. The integrated communication system facilitates real-time updates, showcasing a data flow for direct interaction. Detailed service monitoring involves data flows for accessing project documentation and providing feedback. The resource center and account management tools contribute to an enriched user experience, utilizing distinct data flows. The secure logout option involves a data flow for session termination, and an optional feedback survey triggers a data flow for user input, completing a seamless and user-friendly process that encourages customer retention for future services.

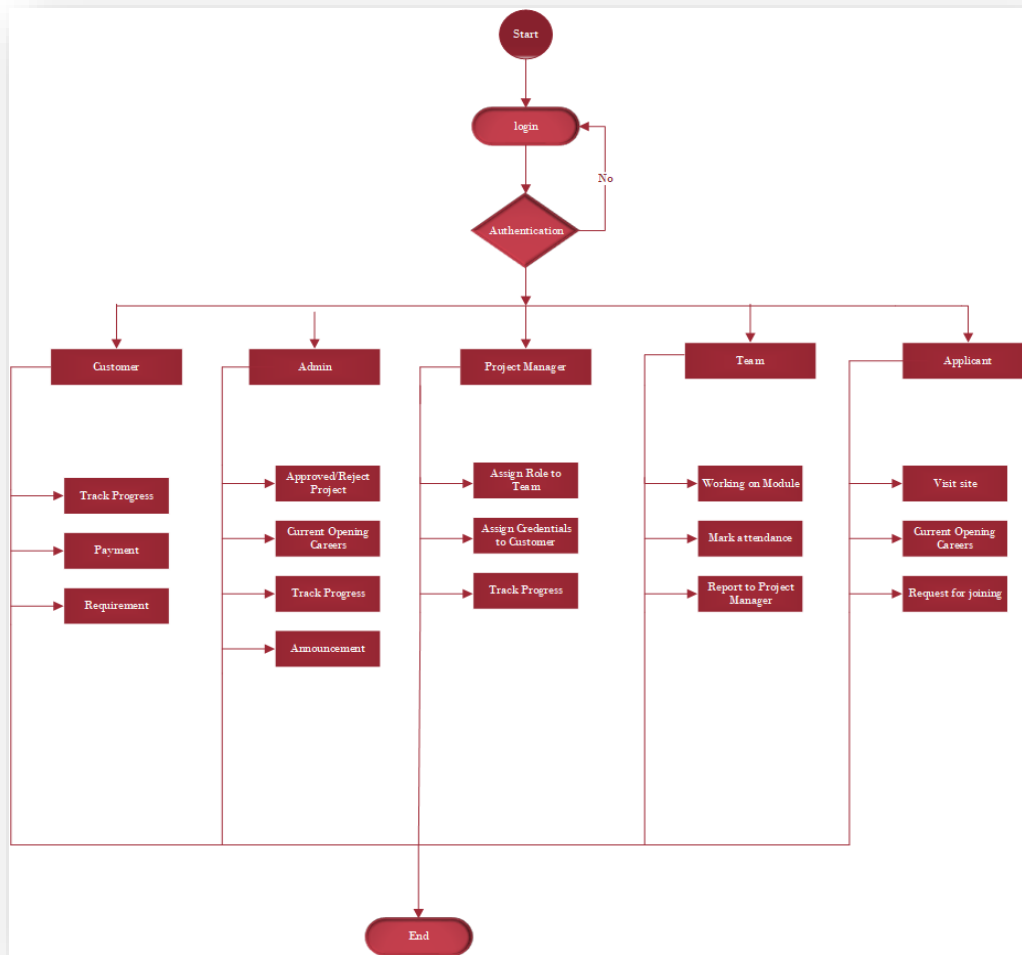


Figure 4.1: Data Flow Diagram of SHMS

4.3 Data Representation

4.3.1 Sequence Diagram

This image shows how the system will work, task after task sequence-wise. It gives a complete description of the workings of our system. In the Software House Management System website, the customer initiates the project lifecycle by submitting a detailed service request, specifying requirements and scope. The admin reviews the proposal, collaborates with the project manager for feasibility, and, upon acceptance, triggers project commencement. The project manager strategically divides tasks among specialized teams, ensuring efficient workflow. Through a dedicated login, the customer gains real-time access to a personalized dashboard, monitoring project progress and accessing documentation. As teams deliver modules, the project manager showcases them on the customer's dashboard, fostering early feedback. Upon customer approval, an integrated payment process is initiated, leading to the project's final delivery. The customer receives confirmation, providing an opportunity for feedback, thereby emphasizing

transparency, collaboration, and iterative development for optimal customer satisfaction.

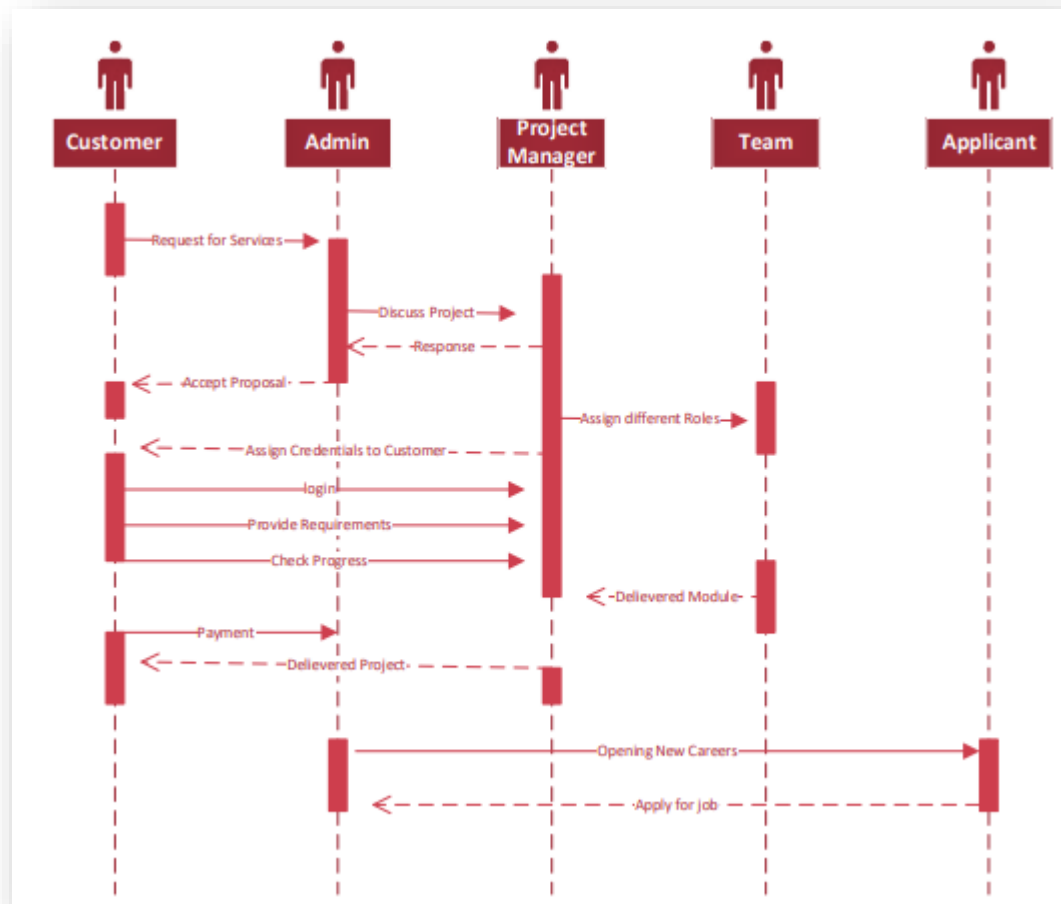


Figure 4.2: Sequence Diagram of SHMS

4.3.2 Class Diagram

The Class Diagram for the Software House Management System delineates the key classes corresponding to distinct user roles. The "Admin" class encompasses functionalities related to overall system administration, with attributes and methods for user management and system configuration. The "Project Manager" class encapsulates features such as project planning, team assignment, and progress tracking, exhibiting attributes and methods specific to project management. The "Customer" class represents users engaging with the system to request and monitor services, with attributes reflecting customer details and methods for service interaction. The "Applicant" class includes attributes and methods pertinent to individuals applying for positions within the software house. Lastly, the "Team" class incorporates

attributes related to team composition and collaboration, serving as the foundational unit for project execution. These classes collectively form a comprehensive Class Diagram, illustrating the system's organizational structure and relationships among different user roles.

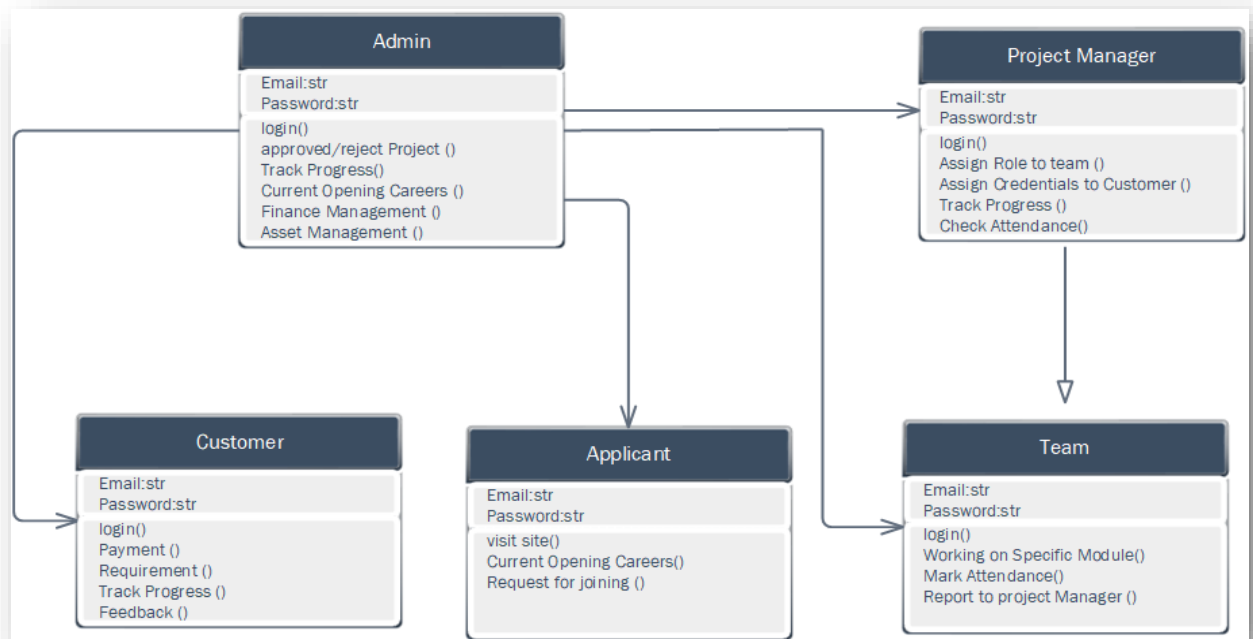


Figure 4.3: Class Diagram of SHMS

Chapter 5

Implementation

5. Implementation

The Software House Management System is implemented through a series of interconnected modules, each addressing specific aspects of software house operations. The overall implementation employs a web-based architecture using the Django framework, with PostgreSQL and SQLite databases. This design choice ensures a robust, scalable, and flexible system capable of efficiently handling the complex challenges faced by software development companies. Together, these modules contribute to a cohesive Software House Management System that enhances productivity, collaboration, and decision-making within the software house.

- The Project Management System provides a centralized platform for administrators to oversee and manage the entire project lifecycle. It empowers administrators to create, modify, and control access to projects, allocate projects among development teams, and maintain comprehensive records of project status, milestones, and timelines. The system fosters effective communication, collaboration, and performance analysis, supporting decision-making and resource allocation.
- A role-based system to organize the workforce efficiently, ensuring specialized teams contribute to overall organizational success. Different teams, such as Web Development and App Development, are structured based on specific roles and responsibilities. Subdivisions within teams, such as Front-End Developers and Back-End Developers, encourage collaboration and specialization. The role-based system facilitates efficient project assignment, ongoing skill development, and organizational flexibility.
- The Attendance Management System contributes to workforce management by providing a systematic way to monitor and record employee attendance. It operates on a two-times-a-day attendance marking system, leveraging technology for accurate tracking. The system includes features like biometric authentication, real-time tracking, automated reports, and integration with the leave management module. Its user-friendly interface encourages consistent and accurate attendance recording.
- The Financial Management System is a vital tool for administrators to monitor, analyze, and manage the financial aspects of the organization. It integrates comprehensive financial statements, calculates total revenue, tracks project-related expenses, generates Profit and Loss statements, supports financial planning through budgeting and forecasting, manages invoicing, and provides customization options for financial reports. The system ensures security and confidentiality of sensitive financial information.

- The Asset Management System is a comprehensive tool for administrators to manage, monitor, and optimize the lifecycle of assets. It serves as a centralized repository, categorizes assets, assigns unique identifiers, supports lifecycle management, integrates maintenance schedules, tracks movement history, and calculates depreciation for tangible assets. The system is seamlessly integrated with the Financial Management module, ensuring accurate financial representation of assets.

5.1 Algorithm and Technologies

- HTML
- CSS
- JAVASCRIPT
- JQUERY
- Bootstrap
- Python
- Django
- PostgreSQL
- Data Gathering Techniques
- Data Preprocessing
- Data Analysis

5.2 User Interface



Figure 5.1: Home Screen of SHMS

Figure 5.1 shows home screen of the Software House Management System which serves as a central hub, offering an engaging and informative interface for users. It is designed to provide a seamless initiation point, featuring key elements that guide users through the system's functionalities. The home screen prominently displays an intuitive navigation menu, facilitating easy access to core modules such as user registration, service requests, and project monitoring. Users are welcomed with an aesthetically pleasing layout, reinforcing a positive user experience. Clear and concise call-to-action buttons prompt users to initiate the signup process, ensuring a streamlined journey. Additionally, the home screen may showcase highlights such as ongoing projects, recent updates, and quick links to essential resources, enhancing user engagement and efficiency. Overall, the home screen acts as a gateway, setting the tone for an efficient and user-friendly interaction with the Software House Management System.

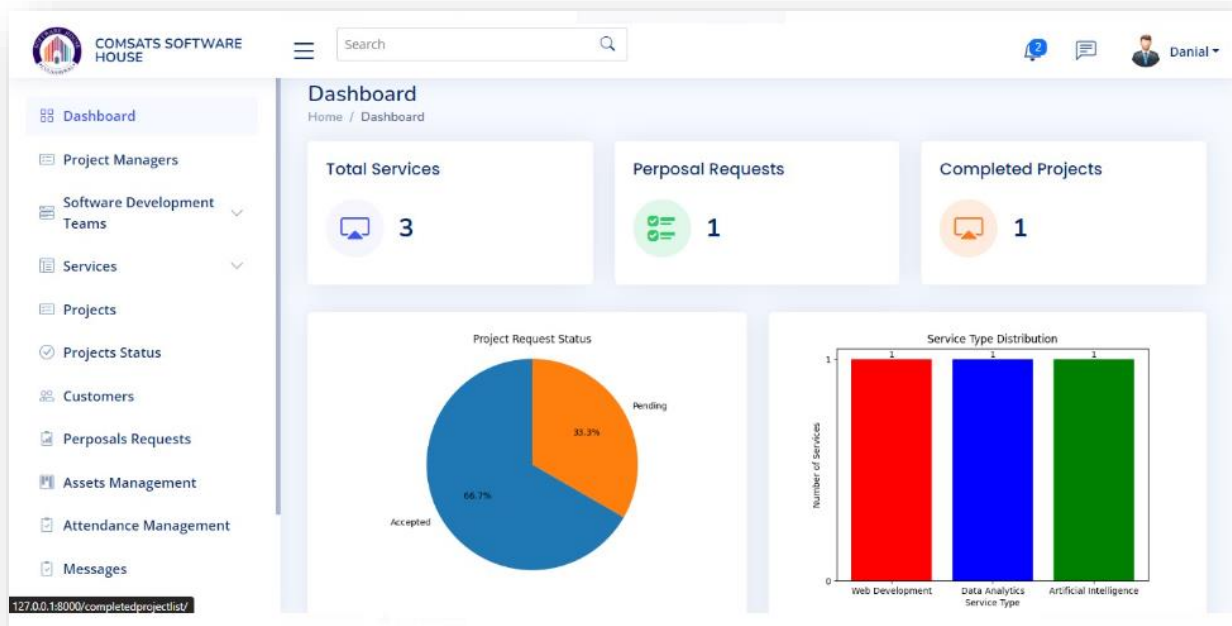


Figure 5.2: Project Dashboard of SHMS

Figure 5.2: Project Dashboard of SHMS Shows the project dashboard screen within the Software House Management System offers users a comprehensive and personalized overview of their ongoing projects. It serves as a dynamic focal point, presenting real-time information on current service statuses, project milestones, and team progress. The dashboard intelligently organizes project-related data, providing users, including project managers and customers, with a visually intuitive representation of key metrics. Users can conveniently track project timelines, view team members' contributions, and access detailed project documentation, fostering efficient project management. Interactive elements enable direct engagement, allowing users to submit feedback, communicate with team members, and make informed decisions based on the presented insights. The project dashboard screen is a pivotal component, enhancing transparency, collaboration, and decision-making throughout the software development life cycle.

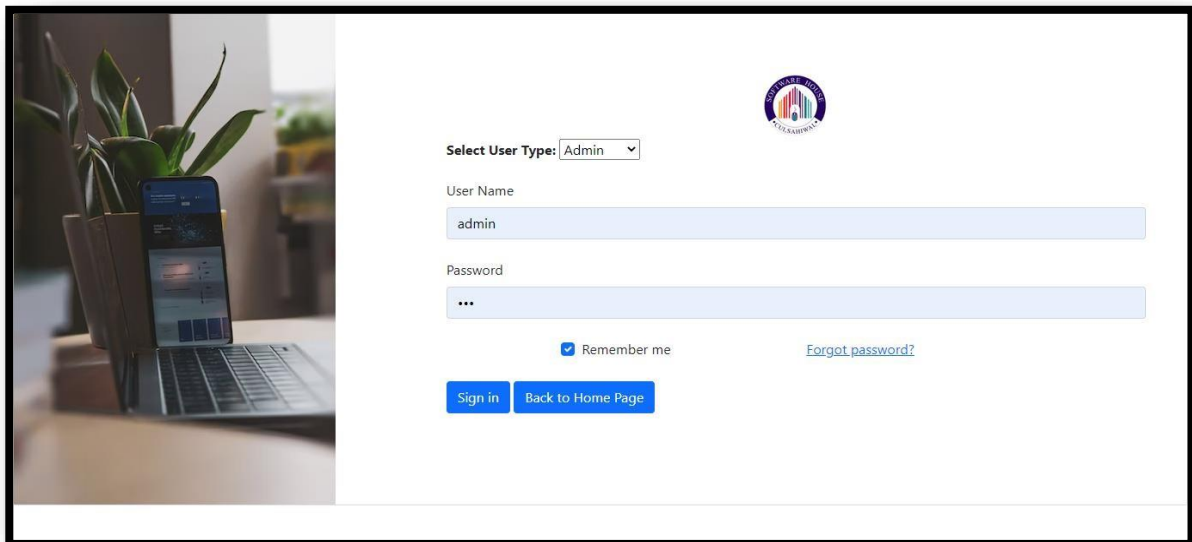


Figure 5.3: login screen of SHMS

Figure 5.3: login screen of SHMS explains the login screen in the Software House Management System serves as the secure gateway for authorized users to access their accounts. It presents a straightforward and user-friendly interface, prompting individuals to enter their credentials for authentication. The screen includes essential elements such as username and password fields, adhering to security best practices to protect user accounts. Upon successful login, users gain access to a personalized dashboard, tailored to their role within the software house. The login screen is designed with a focus on both security and efficiency, ensuring that only authenticated users can navigate the system, providing a secure foundation for personalized and seamless interaction with the Software House Management System.

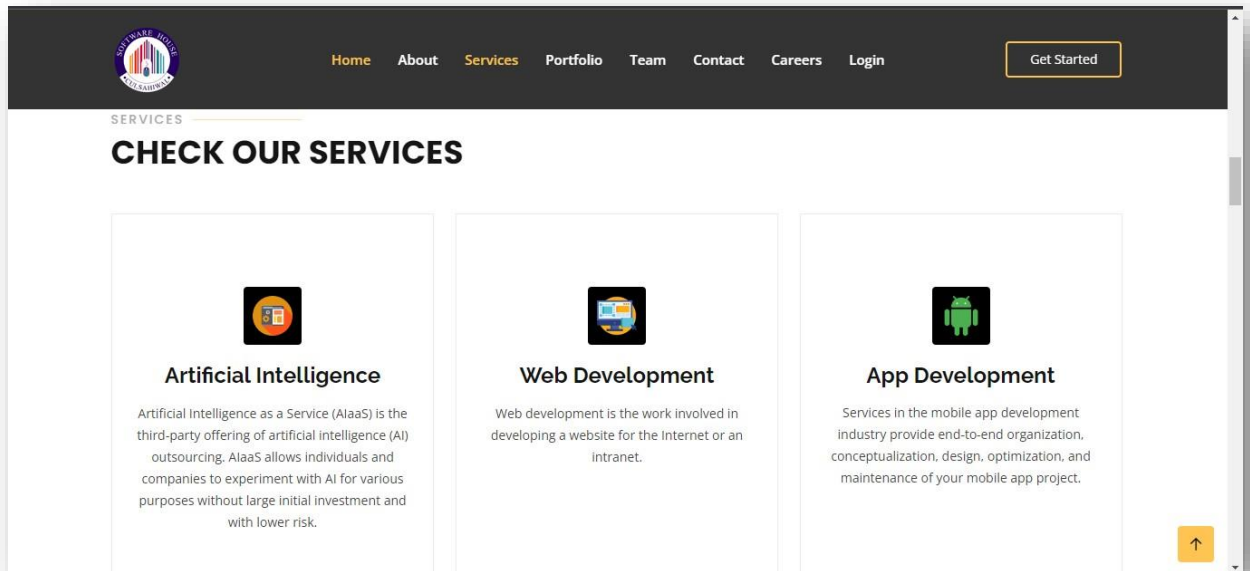


Figure 5.4: Services Interface of SHMS

Figure 5.4: Services Interface of SHMS elaborate the services screen within a Software House Management System plays a pivotal role in orchestrating a diverse array of offerings, encompassing services such as web and app development. This interface serves as a centralized platform where administrators can seamlessly manage and showcase various services, including detailed descriptions, pricing structures, and project durations for web and app development. It empowers the software house to efficiently organize its service catalog, facilitating quick updates and modifications. This screen not only streamlines internal processes for staff but also enhances client interactions by presenting a comprehensive overview of available services. Whether it's outlining the intricacies of web development or specifying the nuances of app creation, the services screen serves as a crucial component in the efficient functioning of a software house, contributing to transparent communication and effective project management.

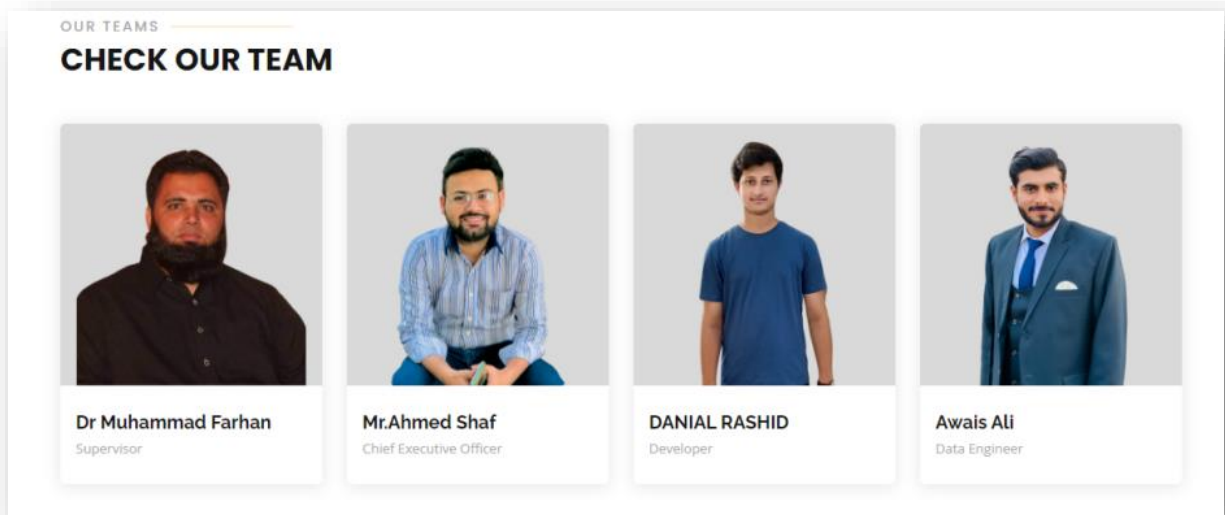


Figure 5.5: Team Interface of SHMS

Figure 5.5: Team Interface of SHMS shows the team screen in a Software House Management System serves as a central hub for overseeing and managing the diverse talents and roles within the organization. This interface typically provides administrators with a comprehensive view of team members, including their roles, expertise, and current project assignments. It enables efficient team collaboration by facilitating the allocation of resources, tracking workloads, and ensuring that the right skills are matched with specific projects.

Chapter 6

Testing and Evaluation

6. Testing and Evaluation

Table 6.1: Project Management Test Cases

Test Case ID	Description	Expected Result	Pass/Fail
PM_TC_01	Create a new project	Project is created successfully	Pass
PM_TC_02	Edit project details	Project details are updated successfully	Pass
PM_TC_03	Assign a team to a project	Team is assigned to the project	Pass
PM_TC_04	View project details	Project details are displayed correctly	Pass
PM_TC_05	Mark a project as completed	Project status is updated to 'Completed'	Pass

Table 6.2: Role Based System Test Cases

Test Case ID	Description	Expected Result	Pass/Fail
RB_TC_01	Create a new user with a specific role	User is created with the specified role	Pass
RB_TC_02	Edit user details and role	User details and role are updated successfully	Pass
RB_TC_03	Remove a user	User is removed from the system	Pass
RB_TC_04	Assign roles to users	Users have the correct roles assigned	Pass
RB_TC_05	Verify role-based access permissions	Users can access only allowed functionalities	Pass

Table 6.3: Attendance Management Test Cases

Test Case ID	Description	Expected Result	Pass/Fail
AM_TC_01	Record attendance for an employee	Attendance record is saved successfully	Pass
AM_TC_02	Edit attendance record for an employee	Attendance record is updated successfully	Pass
AM_TC_03	Generate attendance reports	Reports show accurate attendance data	Pass

Table 6.4: Finance Management Test Cases

Test Case ID	Description	Expected Result	Pass/Fail
FM_TC_01	Create an invoice	Invoice is generated successfully	Pass
FM_TC_02	Record expenses	Expenses are recorded accurately	Pass
FM_TC_03	Generate financial reports	Reports display accurate financial data	Pass
FM_TC_04	Track income and expenses over time	Income and expenses are tracked correctly	Pass
FM_TC_05	Calculate profit and loss	Profit and loss calculations are accurate	Pass

Table 6.5: Asset Management Test Cases

Test Case ID	Description	Expected Result	Pass/Fail
AMM_TC_01	Add a new asset to	Asset is added to the	Pass

	the system	inventory	
AMM_TC_02	Edit asset details	Asset details are updated successfully	Pass
AMM_TC_03	Assign assets to employees or projects	Assets are assigned correctly	Pass
AMM_TC_04	Track asset depreciation	Asset depreciation is calculated accurately	Pass
AMM_TC_05	Generate asset reports	Reports show accurate information about assets	Pass

Chapter 7

Conclusion and Future Work

7. Conclusion and Future Work

7.1 Conclusion

After In conclusion, the Software House Management System represents a significant step forward in addressing the complex challenges faced by software development companies. By providing a comprehensive and integrated solution, this system streamlines various aspects of software house operations, including project management, employee administration, financial tracking, and asset management. Throughout the development process, the project team worked diligently to create a web-based application with robust functionality. The system offers benefits such as efficient project coordination, simplified employee management, accurate financial insights, and optimal asset utilization. Moreover, it enhances security and scalability while promoting user-friendly interaction through its intuitive interface. The implementation of the Software House Management System signifies a commitment to improving operational efficiency and decision-making within software houses. As a result, it contributes to enhanced productivity, reduced administrative overhead, and better resource allocation, ultimately driving the success of software development endeavors.[7]

7.2Future Work

The Software House Management System, while achieving its initial objectives, opens doors to continuous improvement and expansion. The following areas represent potential future work to enhance the system's functionality and adaptability.

Advanced Reporting: it provides users with more in-depth insights into project performance, financial trends, and resource utilization. Integrate advanced analytics and data visualization tools, enabling users to generate detailed reports with customizable metrics and visual representations.

Integration with Third-party Tools: it streamlines workflows and enhance compatibility with existing software in the software house's ecosystem. Explore APIs and integration options with popular project management, accounting, and communication tools such as Jira, QuickBooks, and Slack.

Artificial Intelligence and Machine Learning: it leverages AI and ML for predictive analytics, resource optimization, and automation of routine tasks. Research and implement algorithms that can predict project delivery times, optimize resource allocation based on historical data, and automate repetitive tasks such as data entry and analysis.

Mobile App: it provides on-the-go access for users, especially project managers and field employees, facilitating quick updates and approvals. Develop a mobile application with a user-friendly interface, allowing users to access key functionalities, receive real-time updates, and perform essential tasks from their mobile devices.

Enhanced Security Measures: it continuously fortifies the system against evolving cyber threats and ensure compliance with data privacy regulations. Regularly update and enhance security protocols, conduct security audits, and stay informed about the latest cybersecurity trends to promptly address potential vulnerabilities.

Feedback Mechanism: we gather user suggestions and insights for ongoing improvements and user-driven enhancements. Introduce a feedback mechanism within the system, such as surveys, suggestion boxes, or an integrated communication channel, to collect user feedback and prioritize feature requests.

8. References

- [1] P. Letouze, R. A. Ronzani, and A. H. Oliveira, “An academic project management web system developed through a software house simulation in a classroom,” in Proc. of 2011 International Conference on Sociality and Economics Development, 2011. Accessed: Dec. 28, 2023. [Online].
- [2] Q. I. Sarhan, B. S. Ahmed, M. Bures, and K. Z. Zamli, “Software module clustering: An in-depth literature analysis,” IEEE Transactions on Software Engineering, vol. 48, no. 6, pp. 1905–1928, 2020.
- [3] B. Boehm, J. A. Lane, S. Koolmanojwong, and R. N. Turner, The incremental commitment spiral model: Principles and practices for successful systems and software. Addison-Wesley Professional, 2014. Accessed: Dec. 28, 2023. [Online].
- [4] E. Alba and J. F. Chicano, “Software project management with GAs,” Information sciences, vol. 177, no. 11, pp. 2380–2401, 2007.
- [5] C. Jones, “Software project management practices: Failure versus success,” CrossTalk: The Journal of Defense Software Engineering, vol. 17, no. 10, pp. 5–9, 2004.
- [6] J. Jurison, “Software project management: the manager’s view,” Communications of the association for information Systems, vol. 2, no. 1, p. 17, 1999.
- [7] Stellman and J. Greene, Applied software project management. O’Reilly Media, Inc., 2005. Accessed: Dec. 28, 2023. [Online].